

Monitoring in Multi-Tenant Environments with LDMS

LDMS User Group meeting 1-26-26
Open Grid Computing

Nichamon Naksinehaboon
nichamon@ogc.us
Open Grid Computing

Outlines

- Background
- What are tenants?
- Collect job information
- Collect monitoring data
- Q & A

Background

- Shared node scheduling becoming more common
 - Containerized environments (Kubernetes)
 - Multiple jobs on bare metal (efficient modern CPUs/GPUs)
 - Different jobs may be run by different users
- At a given time, multiple jobs may run on a node

Multiple jobs on a node and multiple users on a node

What are Tenants?

- “Tenant” means different things to different people
- In our scope, we focus on
 - Jobs scheduled on a node
 - Users who schedule the jobs
 - Groups users belongs to

Collect Job Information in Multi-Tenant Environments

Existing Tools: Job Sampler Plugins

Recommended: slurm_sampler2

- For sites using the Slurm job scheduler
- Creates a set with job information from Slurm

```
$ ldms_ls -x sock -p 10001 -a munge -h node-1 -l -E jobs
node-1/jobs: consistent, last update: Fri Jan 23 10:05:58 2026 -0600 [818960us]
D u64      component_id      0
M record_type job_data      LDMS_V_RECORD_TYPE
M record_type task_data      LDMS_V_RECORD_TYPE
D list<>    job_list
  job_id app_id  user      job_name job_tag job_state job_size job_uid job_gid job_start (timestamp) job_end (timestamp) node_count task_count
    23     0 "user-1" "interactive" ""      2        1    1000    1000    1769184357          0          1          1
    24     0 "user-1" "interactive" ""      2        1    1000    1000    1769184358          0          1          1
D list<>    task_list
  job_id task_pid task_rank task_exit_status
    23    3950      0          0
    24    3988      0          0
```

Coming soon: Multi-Tenant Feature

- **Limitation of existing job samplers**

- Fixed combination of job attributes
- Tags system-samplers with the same data that a job-specific sampler is tagged

- **Multi-Tenancy**

- Through configuration, a “tenant” is defined
 - The configuration specifies which attributes constitute this “tenant”
- A tenant configuration is assigned to a metric set, called tenant set
- Metric sets are tagged with a UUID that can be used to recover the configured data

- **Flexibility**

- Samplers can be configured to associate tenant metrics with its data through the tenant configuration
- Different sampler types may choose different tenant configurations and therefore different tenant metrics
- Different sites can track different combinations of job attributes (job-level, user-level, step-level)

Different Job Schedulers Across Sites

- Sites use different schedulers
 - Some use Slurm
 - Some use Flux or Flux + Slurm
 - Some may use others
- Challenge: Each has different attribute names/availability
- Solution: Idmsd provides a uniform scheduler interface
 - Same attribute names regardless of scheduler
 - Initial support: Slurm and Flux
 - Users can reuse tenant configurations across systems with different schedulers

Uniform Job Attributes Across Schedulers

job_id	step_id	task_id
user	step_start	task_pid
job_name	step_end	task_rank
job_uid		task_start
job_gid	node_count	task_end
job_start	total_tasks	task_exit_status
job_end	local_tasks	
job_state		

Multi-Tenant Feature: Define Tenants

- Define a tenant definition
 - `tenant_add name=my_tenant attributes=job_id,step_id,task_id,uid,gid`
- A tenant set containing a list of entries will be created

```
tenant_add name=jobs  
          attributes=job_id,job_uid,job_gid,job_start,job_end
```

Multiple tenant definitions can be added

```
tenant_add name=job_step_task  
          attributes=job_id,job_uid,job_gid,step_id,task_id,job_start,job_end
```

Example tenant sets: 2 Jobs having 1 step each

```
$ ldms_ls -x sock -p 10001 -a munge -h node-1 -l -E tenant
node-1/my_tenant: consistent, last update: Fri Jan 23 10:05:58 2026 -0600 [818983us]
D char[]      last_event      "4"
M record_type jobmgr_query_rec LDMS_V_RECORD_TYPE
D list<>      tenants
      job_id job_uid job_gid step_id task_id      job_start job_end
"slurm_23"   1000    1000 "step-0" "task-0" 1769184357.000000 0.000000
"slurm_24"   1000    1000 "step-0" "task-0" 1769184358.000000 0.000000
```

Example tenant sets: 2 Jobs & 1 job having 2 step

```
$ ldms_ls -x sock -p 10001 -a munge -h node-1 -l -E tenant
node-1/my_tenant: consistent, last update: Fri Jan 23 10:06:59 2026 -0600 [50007us]
D char[]      last_event      "4"
M record_type jobmgr_query_rec LDMS_V_RECORD_TYPE
D list<>      tenants
      job_id job_uid job_gid step_id task_id      job_start      job_end
"slurm_23"   1000    1000 "step-0" "task-0" 1769184357.000000 1769184417.000000
"slurm_24"   1000    1000 "step-0" "task-0" 1769184358.000000 0.000000
"slurm_24"   1000    1000 "step-1" "task-0" 1769184358.000000 0.000000
```

key: 123456

```
meminfo = {
  tenant_key: 123456,
  memtotal,
  memfree,
  ....
}
```

Specify Job Scheduler

- Enable Message Bus
- load, config, and start the job scheduler plugin (jobmgr_slurm)
- Add tenant definitions

```
1 # Enable Messages
2 msg_enable
3
4 # Start jobmgr_slurm to receive Spank notifier messages
5 load name=jobmgr_slurm
6 config name=jobmgr_slurm
7 jobmgr_start name=jobmgr_slurm
8
9 # Tenants
10 tenant_add name=my_tenant metric=job_id,job_uid,job_gid,step_id,task_id,job_start,job_end
11
```

Collect Monitoring Data in Multi-Tenant Environments

Current LDMS Monitoring

- LDMS collects system-wide metrics today
- Examples: /proc/meminfo, /proc/vmstat, /proc/stat, /proc/net/dev
- These measure node-level resource usage
 - Node has 80% CPU in use and 8GB free memory

Multi-tenant Challenges

- Today: Single job_id tagged to per-system metric sets (existing sets)
 - Works when one job owns the node
 - Per-system metrics = job consumption (node totals = job totals)
- Multiple jobs per node:
 - Existing metric sets still measure node totals: “Node used 80% CPU”
 - Per-job questions ask: “How much did Job 123 use?”
 - Node totals $\neq$ individual job consumption
 - Tagging multiple job_ids doesn’t provide answers to the per-job questions
- Real-world scenario:
 - “My job ran in 10 minutes last Tuesday, but 20 minutes this Tuesday - why?”
 - Node totals don’t reveal: Job A used 70% CPU, Job B used 25% CPU
 - Without per-job data, can’t identify resource contention

Two Types of Monitoring Data

- **Per-system metrics: (What LDMS collects today)**
 - Node-level resource measurements
 - Example: meminfo, vmstat, procstat
 - Measures: “Node has 80% CPU in use and 8GB free memory”
- **Per-job metrics: (What we don’t collect yet)**
 - Resource consumption by individual jobs
 - Measure: “Job 123 used 45% CPU, consumed 2 GB memory”

These are fundamentally different types of data

Why These Require Different Approaches

- System metrics measure node state
- Per-job metrics measure individual consumption
- Tagging multiple job_ids to system metrics doesn't answer per-job questions

These require different collection and tagging approaches

Our Goals

Enable **end users** and **scientists** to efficiently query:

1. **Give me metrics for my jobs on Tuesday**
2. **Give me metrics for my jobs for application “xyz”**
3. **Give me metrics for my job of job-id “X”**

Job metadata queries:

4. **What were my jobs on Tuesday?**
5. **What were my ‘xyz’ jobs between Monday and Friday**

Performance analysis:

6. **Why was my job slower this week than last week?**
 - **Was it co-scheduled with a resource-intensive job?**
 - **Were there resource collisions?**

Status

- Job metadata queries (queries 4-5)
 - Which jobs ran, when
 - Enabled by **job sets** and **tenant sets**
- Per-job resource queries (queries 1-3 and 6)
 - How much did each job consume
 - **Requires per-job collection**

Per-Job Resource Collection

Currently exploring approaches to collect actual per-job resource usage:

- /proc/<pid> based collection
- cgroup-based collection

Each has different trade-offs in:

- Data availability
- Configuration requirements
- LDMS performance characteristics

Per-Job Resource Collection: /proc/pid

- Collect data per process, aggregate to job/step/task levels
- Available metrics per process:
 - CPU time (user, system)
 - Memory usage (RSS, virtual)
 - I/O counters (read/write bytes, operations)
- Pros:
 - Available on every Linux system
 - No special configuration required
 - Works with any job scheduler
- Cons:
 - Must track PIDs for job processes
 - LDMS calculates aggregation per job/step/task

Per-Job Resource Collection: cgroup-based

- Resource usage at different levels readily available (job/step/task)
- Available data (varies by cgroup version and job scheduler configuration)
 - CPU usage and throttling
 - Memory usage, limits, and OOM events
 - Block I/O statistics (reads/writes)
- Pros:
 - Efficient aggregation by kernel
 - Captures all processes in cgroup
- Cons:
 - Depends on cgroup configuration
 - Depends heavily on job scheduler configuration and integration

What Queries Matter to You?

- Our current query goals (recap):
 1. Give me my job data on Tuesday
 2. Give me my job data for application "xyz"
 3. Give me my job data for job-id "X"
 4. What were my jobs on Tuesday?
 5. What were my 'xyz' jobs between Monday and Friday
 6. Why was my job slower this week than last week?
- Questions for the community:
 - What other queries are critical for your site?
 - What else should we be able to answer?
 - What would make per-job analysis easier for you?

Q & A

- Next Multi-tenant discussion is Feb 9 at the User Group Meeting

Q & A

- Aiden (LANL) brought up a concern of high overhead if all metric sets contains the list of tenants. This leads to data duplication.
 - Planned solution: The plan is to tag each metric set with a UUID of the tenant. The UUID works as the key to identify which tenant the set associated with.
 - For example, a tenant definition, job_tenants, is the combination of job_id, uid, gid and the key is 12345. Each metric set can be tagged with the key 12345.
job_tenants set = { ..., key : 12345, tenants : [{1, 1000, 1000, start_time, end_time},
{2, 1002, 1002, start_time, end_time}} }
meminfo set = { ..., tenant_key : 12345, memfree: 1.5TB, }
- Mike (LANL) brought up that in his environment, when a user submits a job, a user/job environment is created (PID namespace, user namespace, limit gid, mount namespace, ..) to limit resource and isolate the user-job environment from the other environment. Inside the job environment, pid, for example, is different from the PID in the global (host) environment.
 - In Idmsd world, how to map the inside information and the global information?
 - Tom brought up security questions of what information the job environment can share with the global environment and with other job environment
- Deb (LANL) what to be able to answer the question “how efficiently users’ job utilize the gpu allotted to it?”
 - Metrics in the dcgm sampler was mentioned as a measurement of efficiency?

Q & A

- John (LBL) want to be able to collect memory, cpu, network, gpu metrics **per-job**

Thank you!